

Dokumentation & Projekttagebuch

Innovation Lab 3
Jahr 2026

Projekt: **IDERHA**

Team: Nr. **23**

Inhalt

1. Allgemeine Informationen	3
1.1 Management-Summary des Projektes	3
1.2 Rahmenbedingungen und Projektumfeld	3
1.3 Semester-Roadmap:	4
1.4 Collaboration & Tooling:	4
1.5 Anmerkungen:	4
2. Projekt-Kurzbeschreibung	5
2.1 Deliverables:	5
2.2 Herausforderungen:	5
2.3 Mehrwert für Anwender*innen:	5
2.4 Scope & Nicht-Ziele:	5
3. Spezifikation der Lösung	6
3.1 Systemumfeld	6
3.2 Features / Funktionale Anforderungen WS25	6
3.3 Schnittstellen	7
3.4 Qualitätseigenschaften	7
3.5 Sonstige Merkmale	7
3.6 UI	7
3.7 Sequenzdiagramme	10
4. Aufwandschätzung	11
4.1 Aufwandschätzung mittels Delphi-Methode	11
4.2 Aufwandschätzung mit Zugesicherung	12
5. Auslieferung	13
5.1. Fertige Lösung inklusive Source-Code	13
5.2. Systemarchitektur und Datenhaltung	13
5.3 Lizenzierung und Copyright	14
5.4 Hardware-Anforderungen	14
5.5 Voraussetzungen für Installation und Betrieb	14
5.6 Installation und Betrieb	15
6. Unser Projekt-Tagebuch	15

1. Allgemeine Informationen

Projektname: IDERHA

Supervisor: Lukas Rohatsch

Innovation Lab 3, WS 2025

Projektteam:

Zehinovic Aldin, if22b130@technikum-wien.at - Teamleiter

Dervisefendic Armin, if23b040@technikum-wien.at

*Bitte beachten Sie: Um mögliche Urheberrechtsprobleme zu vermeiden, haben wir uns entschieden, unsere Lösung **eHealth Insights** anstelle von **IDERHA** zu nennen.*

1.1 Management-Summary des Projektes

Das Projekt eHealth Insights ermöglicht die datenschutzkonforme Verwaltung, Analyse und Visualisierung medizinischer Daten aus mehreren Krankenhäusern. Kernziele waren:

1. Aufbau eines sicheren Benutzer- und Rollenmanagements (Admin, Hospital, Researcher).
2. Implementierung dezentraler Krankenhaus-Datenbanken unter zentraler Authentifizierung.
3. Entwicklung eines Analyse- und Visualisierungs-Dashboards.
4. Umsetzung von Exportmöglichkeiten (PDF/DOC) für Analysen.
5. Sicherstellung von Sicherheit, DSGVO-Konformität und performanter Datenverarbeitung.

1.2 Rahmenbedingungen und Projektumfeld

- **Sicherheitsanforderungen:** Verschlüsselte Speicherung, BCrypt-Passwort-Hashing, getrennte Datenbanken für Patientendaten, rollenbasierte Zugriffskontrolle (JWT).
- **Systemumgebung:** Containerisierung mit Docker, Deployment auf Linux-Servern, lokale Entwicklung über Docker Compose.
- **Technologien:** Frontend: React.js (Vite, TailwindCSS), Backend: Spring Boot (Java, Spring Security, JWT), Datenbanken: PostgreSQL nach OMOP-CDM.
- **Qualität:** Benutzerfreundlich, wartbar, performant, skalierbar.
- **Terminvorgaben:** Prototyp und Testumgebung am Ende von Sprint 6 bereitgestellt.

1.3 Semester-Roadmap:

Die Umsetzung erfolgte in 6 Sprints:

Sprint	Datum	Fokus	Erreicht
1	14. October, 2025	API Foundation	Schnittstellen & Key Management umgesetzt
2	4. November, 2025	UI Components & Logic	Charts, Filter, Datenverarbeitung
3	18. November, 2025	Core Database & Federation (High Complexity)	OMOP Setup, FDW, Dashboard implementiert
4	2. Dezember, 2025	Containerization & DevOps	Docker-Container & Deployment-Pipeline stabilisiert
5	16. Dezember, 2025	Usability & Export	Export PDF/DOC & Hospital-Management
6	13. Jänner, 2026	Security & Quality	Passwort-Hashing, Unit-Tests, Integration-Tests, Security Audit

1.4 Collaboration & Tooling:

- **Version Control:** Git & GitHub (<https://github.com/mide553/IDERHA>)
- **Large File Storage:** Git LFS (für SQL-Dumps)
- **Deployment:** Docker Compose
- **Projektmanagement:** GitHub Projects Board
(<https://github.com/users/mide553/projects/1>)
- **Kommunikation:** Zoom
- **IDE:** Visual Studio Code & IntelliJ IDEA

1.5 Anmerkungen:

Alle geplanten Features wurden umgesetzt und getestet. Plattform erfolgreich in Testumgebung deployed, bereit für produktiven Einsatz.

2. Projekt-Kurzbeschreibung

Im Wintersemester 2025/26 wurde die bestehende eHealth Insights Plattform erweitert und fertiggestellt. Die zentralen Ziele waren die sichere Verwaltung, Analyse und Visualisierung medizinischer Daten aus verschiedenen Krankenhäusern.

2.1 Deliverables:

1. **Visualisierung & Reporting:** Integration von Dashboards, Graphen, Diagrammen; Export von Analysen als PDF/DOC.
2. **Backend-Optimierung:** REST-API-Optimierung, Unit- und Integrationstests, Performance-Verbesserung.
3. **Sicherheitsmaßnahmen:** Verschlüsselung, BCrypt-Passwort-Hashing, rollenbasierte Zugriffskontrolle (RBAC).
4. **Qualitätssicherung:** CI/CD-Pipeline, automatisierte Tests, Logging & Monitoring.
5. **Deployment:** Docker-basierte Bereitstellung, Plattform stabil in Testumgebung verfügbar.

2.2 Herausforderungen:

- Performance bei großen Datenmengen und mehreren Krankenhaus-Datenbanken.
- Technische Komplexität von FDW und OMOP-Datenbank.
- DSGVO-Konformität und Security-Audit.

2.3 Mehrwert für Anwender*innen:

- Effiziente Analyse und Visualisierung von Patientendaten.
- Einfacher Zugriff auf Berichte via Export oder Dashboard.
- Intuitive Bedienung für unterschiedliche Nutzerrollen.

2.4 Scope & Nicht-Ziele:

- Scope: Visualisierung, Export, Sicherheit, Performance, Deployment, QA.
- Nicht-Ziele: KI-Module, Real-Time-Streaming, direkte Datenbankmanipulation durch Nutzer.

3. Spezifikation der Lösung

3.1 Systemumfeld

- **Frontend:** React.js Web UI
- **Backend:** Spring Boot REST API mit Authentifizierung & Datenverarbeitung
- **Datenbanken:** PostgreSQL (OMOP-CDM), separate Krankenhaus-DBs
- **Containerisierung:** Docker für DB, Backend & Frontend als Services
- **Schnittstellen:** REST-API zwischen Frontend, Backend, Exportmodul und Krankenhaus-Datenbanken

3.2 Features / Funktionale Anforderungen WS25

Feature	Beschreibung	User Story / Epic
JWT Authentication	JSON-Endpunkte für Skripte	Als Nutzer möchte ich mich sicher per Token einloggen, damit ich nur auf die Endpunkte zugreifen kann, die meiner Rolle erlaubt sind.
BCrypt Password Hashing	CRUD API Keys.	Als Admin möchte ich, dass alle Passwörter und API-Keys sicher gehasht werden, damit sensible Daten geschützt sind.
User Management	BCrypt.	Als Admin möchte ich Benutzer erstellen, bearbeiten und löschen können, damit ich den Zugriff für verschiedene Rollen verwalten kann.
Patient Data API	CRUD-Operationen auf Patientendaten	Als Krankenhaus-Mitarbeiter möchte ich Patientendaten über die API verwalten können, damit ich Patienteninformationen hinzufügen, aktualisieren oder abrufen kann.
FDW Integration	Anbindung externer Krankenhausdatenbanken	Als Forscher möchte ich auf mehrere Krankenhausdatenbanken über eine Schnittstelle zugreifen können, damit ich Daten analysieren kann, ohne sie zu duplizieren.
Analytics API	Gefilterte Patientendaten bereitstellen	Als Forscher möchte ich Patientendaten mit Filtern abfragen können, damit ich aussagekräftige Analysen und Visualisierungen erstellen kann.
Export Reports	PDF & DOC export	Als Forscher möchte ich Analyseergebnisse als PDF oder DOC exportieren können, damit ich Berichte mit Kollegen teilen kann.
Unit & Integration Tests	Backend-Funktionalität überprüft	Als Entwickler möchte ich automatisierte Tests für das Backend haben, damit die Stabilität und Zuverlässigkeit des Systems gewährleistet ist.

3.3 Schnittstellen

- Frontend ↔ Backend: REST-API
- Backend ↔ Krankenhaus-DBs: REST-API über Docker
- Backend ↔ Exportmodul: REST-API

3.4 Qualitätseigenschaften

- Skalierbar, sicher, performant, wartbar, benutzerfreundlich

3.5 Sonstige Merkmale

- Logging & Monitoring für Backend und Container.
- DSGVO-konforme Datenhaltung und Audit-Logging.

3.6 UI

eHealth Insights Sign In

Login to eHealth Insights

Email:

Password:

Sign In

[Privacy Policy](#)
[Terms of Service](#)
[About Us](#)

Email: support@innoproject.com
Phone: +43 123 456 789

© 2024 INNO Team 23

Abbildung 3.6.1: Login-Seite

eHealth Insights
Home
Help
About Us
Analytics
Manage Users
Upload Data
Sign Out

Manage Users

Add New User

Email
Password
First Name
Last Name
Researcher
Add User

Existing Users

Filter by Role: Hospitals

Email: hospital@ehealth.com
Name: Hospital Hospital
Role: hospital
Assigned Database: hospital1
Created By: admin@admin.com
Edit Delete

Email: hospital2@ehealth.com

Abbildung 3.6.2: “Manage Users“-Seite (Admin)

eHealth Insights
Home
Help
About Us
Analytics
Manage Users
Upload Data
Sign Out

Upload Data

Your Assigned Database:
Hospital 2 (Port 5434)
You can only upload data to this database.

Upload SQL File

Choose SQL File
Upload and Execute SQL

Upload Results:

Status: success
Database: Hospital 2 (Port 5434)
Statements Executed: 8
Message: SQL file processed successfully
Execution Details:

TRUNCATE TABLE condition_occurrence CASCADE: Executed successfully
TRUNCATE TABLE concept CASCADE: Executed successfully
TRUNCATE TABLE person CASCADE: Executed successfully

Abbildung 3.6.3: “Upload Data“-Seite (Hospital)

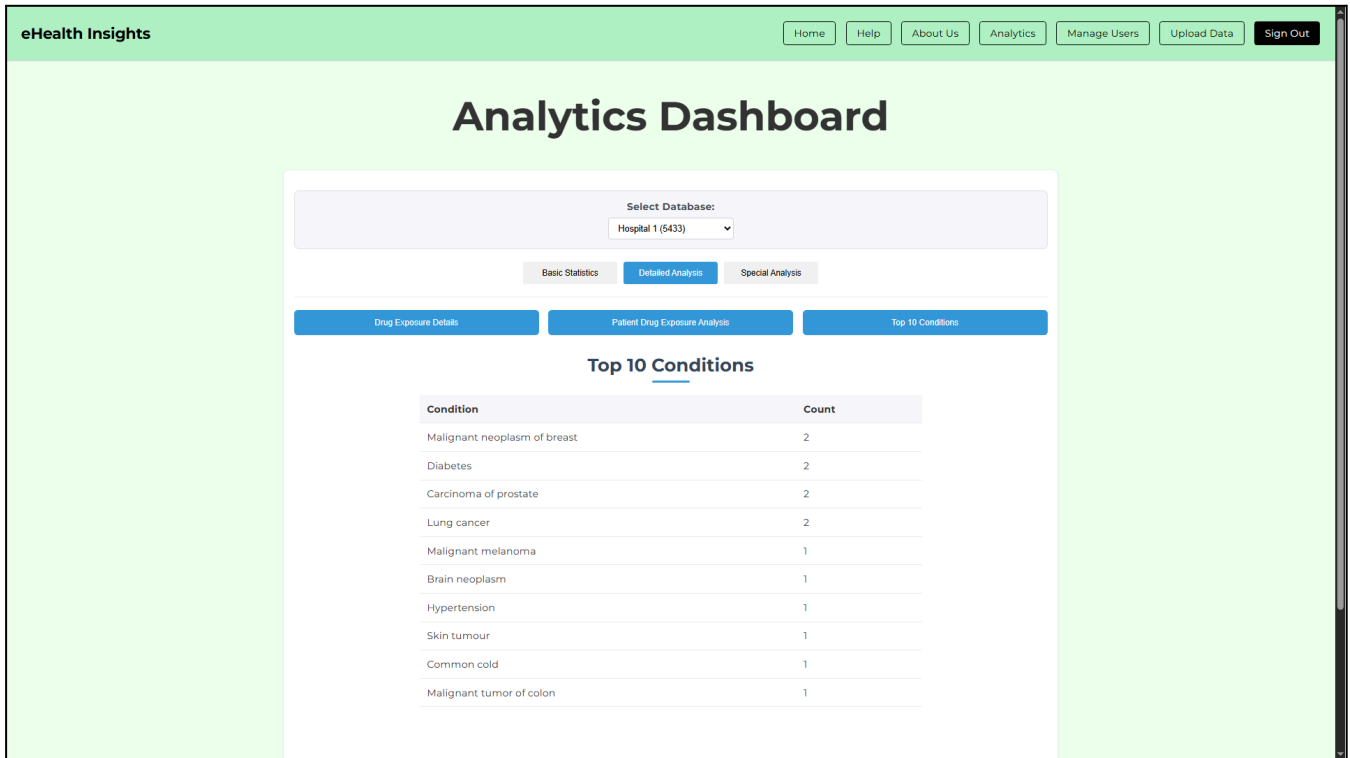


Abbildung 3.6.4: “Analytics Dashboard”-Seite (Researcher)

3.7 Sequenzdiagramme

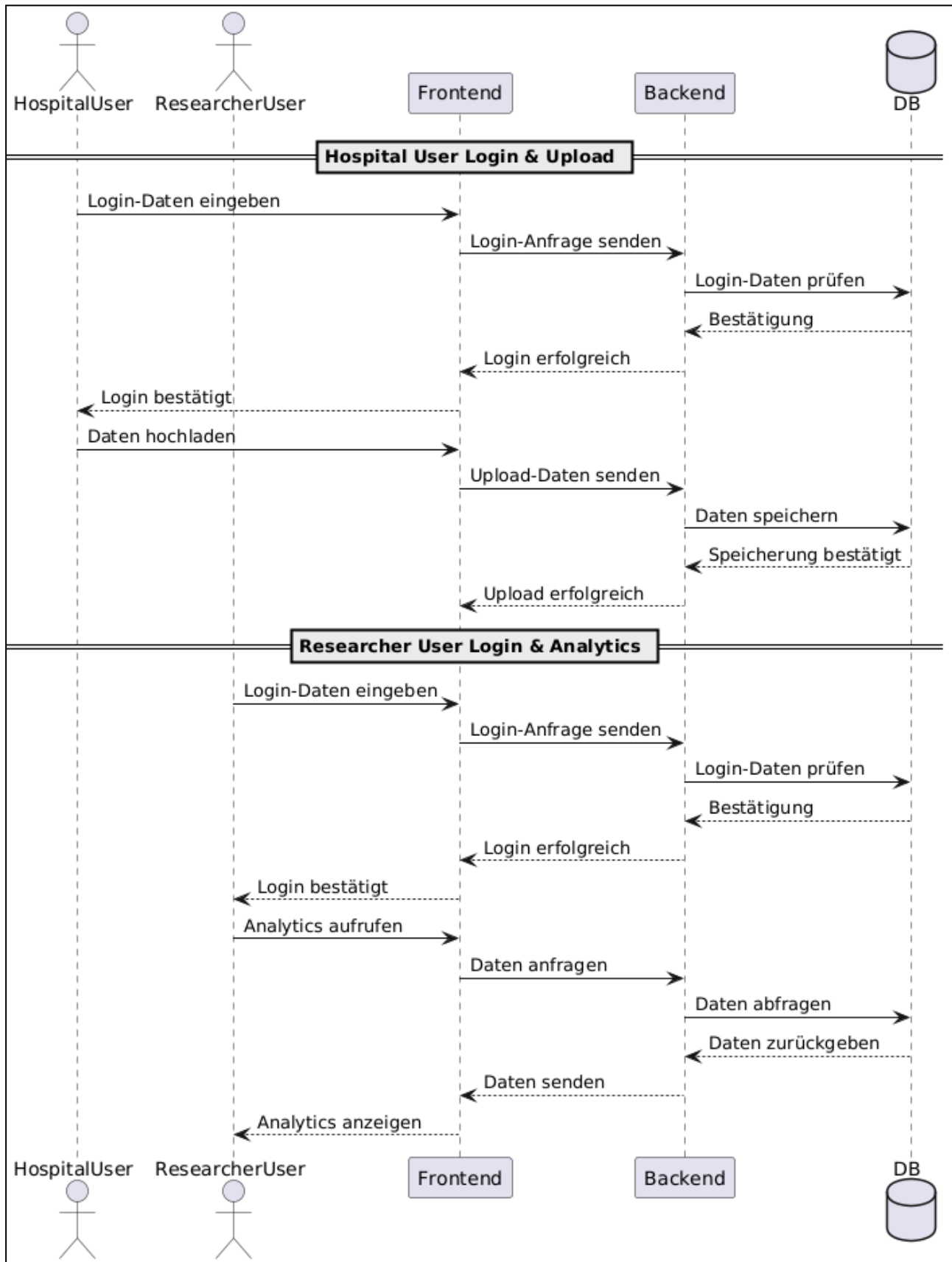


Abbildung 3.7.1: Hospital User Login & Upload und Researcher-User Login & Analytics

4. Aufwandschätzung

4.1 Aufwandschätzung mittels Delphi-Methode

Die Aufwandschätzung für das Wintersemester 2025 (WS25) wurde mithilfe der Delphi-Methode in Kombination mit der PERT-Formel (1:4:1) durchgeführt.

Vorgehensweise:

- Für jede User Story wurden optimistische, wahrscheinliche und pessimistische Schätzwerte in Story Points (Sp) festgelegt.
- Das Team hat diese Werte diskutiert, um Risiken, Annahmen und mögliche Probleme zu dokumentieren (z. B. Performance bei großen Datenmengen, Komplexität von Queries, Usability).
- Auf Basis der PERT-Formel $(O+4M+P)/6$ wurde der erwartete Aufwand für jede User Story berechnet.
- Zur Darstellung der Unsicherheit wurde zusätzlich der Bereich p(range) angegeben.
- Das T-Shirt-Sizing zeigt die relative Größe/Komplexität der Story, der Businesswert den Nutzen für die Anwender*innen.

Beispielhafte Ergebnisse aus WS25:

ID	User Story	Optimistisch	Wahrscheinlich	Pessimistisch	Erwartete PERT	T-Shirt	Businesswert
1	REST-Endpoints für curl/Python	10	14	22	14,67	L	XXL
2	API-Dokumentation mit Beispielen	1	1	3	1,33	S	XL
3	API-Keys verwalten (CRUD)	2	4	8	4,33	M	XL
4	BCrypt-Hashing für Passwörter	3	4	5	4,00	M	XL
5	OMOP-Schema nutzen	8	12	20	12,67	L	XL

Die vollständige Tabelle mit allen User Stories, Schätzungen, Standardabweichungen, Varianz, p(range) und T-Shirt-Sizing ist im zugehörigen Excel-Dokument hinterlegt. Dort können die Werte auch für die Planung der Sprints und die Priorisierung der Aufgaben direkt genutzt werden.

<https://docs.google.com/spreadsheets/d/1HpmTO33xPLkJ9RfgYV90CTHMoV5ExONGpntY9WTrTug/edit?usp=sharing>

4.2 Aufwandschätzung mit Zusagesicherheit

Zur besseren Einschätzung der Verlässlichkeit der Schätzung wurde ein Konfidenzintervall erstellt, das den geschätzten Aufwand in Personenstunden, -tagen, -monaten und -jahren bei unterschiedlichen Wahrscheinlichkeiten zeigt:

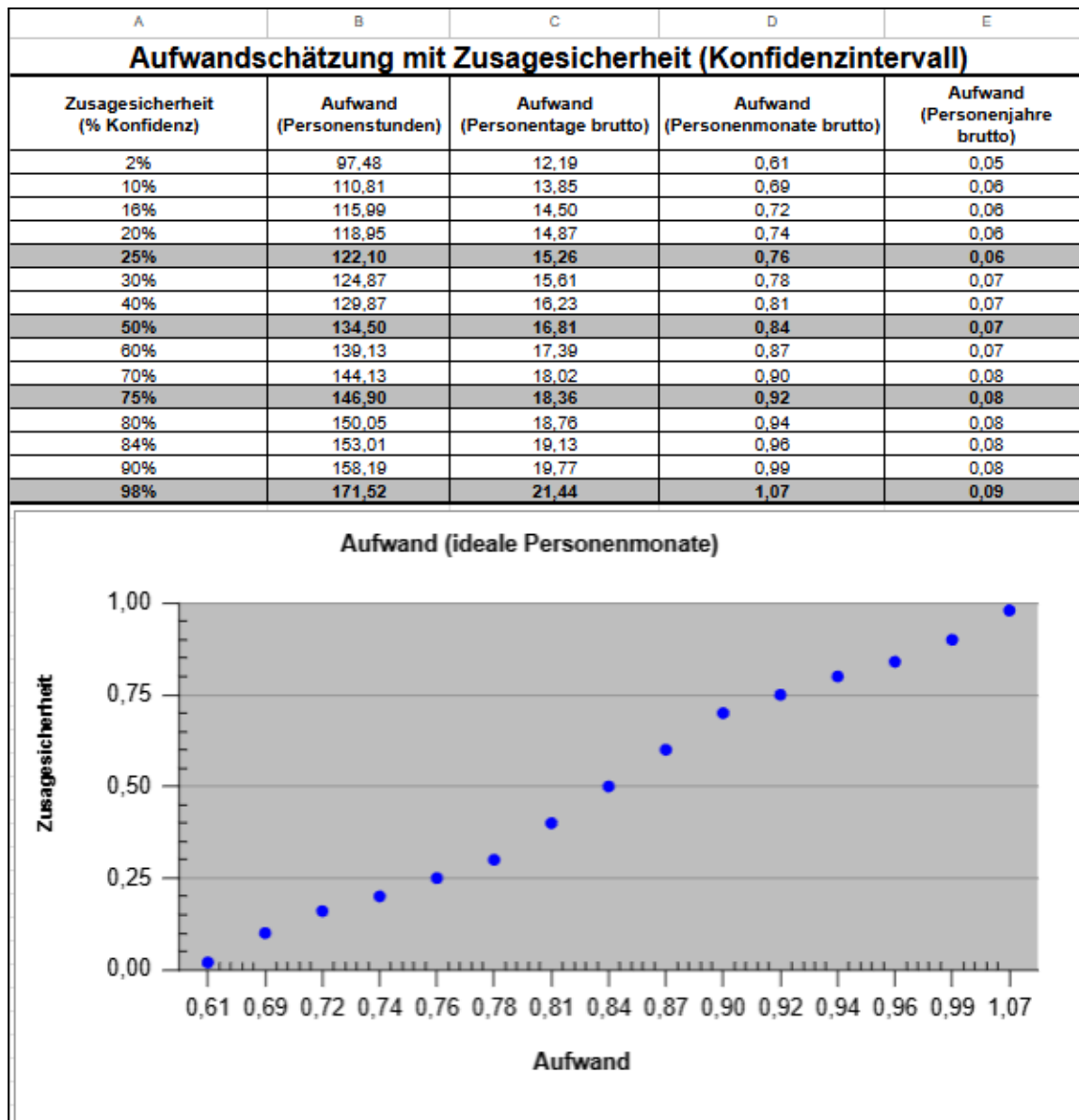


Abbildung 4.1.1: Aufwandschätzung

Bei 50 % Konfidenz beträgt der erwartete Aufwand ca. 16,81 Personentage (~0,84 Personenmonate).

Bei sehr hoher Sicherheit (98 %) sollte man bis zu 21,44 Personentage (~1,07 Personenmonate) einplanen.

5. Auslieferung

5.1. Fertige Lösung inklusive Source-Code

Unsere Lösung besteht aus folgenden Hauptkomponenten:

- **Frontend:** Eine benutzerfreundliche Web-Oberfläche auf Basis von React, die den Zugriff auf Gesundheitsdaten sowie deren Visualisierung ermöglicht.
- **Backend:** Ein RESTful API-Service, implementiert mit Spring Boot, der Daten verarbeitet und mit der zentralen Datenbank sowie externen Services kommuniziert.
- **Datenbank:** Eine zentrale PostgreSQL-Datenbank, die alle gesammelten Gesundheitsdaten speichert und eine effiziente Verarbeitung ermöglicht. Die Datenbankstruktur orientiert sich am OMOP Common Data Model (CDM).
- **Docker-Container:** Die PostgreSQL-Datenbank wird in einem Docker-Container betrieben, um eine einfache Bereitstellung und Verwaltung sicherzustellen.

Der vollständige Source-Code wird im GitHub-Repository (<https://github.com/mide553/IDERHA>) hinterlegt und steht für Weiterentwicklung sowie Wartung bereit.

5.2. Systemarchitektur und Datenhaltung

Die Lösung basiert auf einer modularen Microservice-Architektur, die folgende Struktur aufweist:

- **Frontend:** React-basiertes Web-Frontend, kommuniziert über HTTP mit dem Backend.
- **Backend:** Spring Boot Service mit REST-API-Endpunkten, verwaltet die Logik und Datenverarbeitung.
- **Datenbank:** PostgreSQL-Datenbank, betrieben in einem Docker-Container, mit strukturierter Speicherung nach OMOP-CDM.
- **Containerisierung:** Nur die Datenbank wird mittels Docker betrieben, Frontend und Backend laufen als eigenständige Services.

5.3 Lizenzierung und Copyright

- Die Lösung nutzt Open-Source-Komponenten wie React, Spring Boot und PostgreSQL, jeweils unter ihren jeweiligen Lizenzen (MIT, Apache 2.0, PostgreSQL License).
- Für den Betrieb sind keine zusätzlichen kostenpflichtigen Lizenzen erforderlich.

5.4 Hardware-Anforderungen

- Die Lösung ist für den Betrieb auf Standard-Serverinfrastruktur ausgelegt.
- Empfohlen wird ein Server mit mindestens 4 CPU-Kernen, 8 GB RAM und ausreichend Speicherplatz für Datenhaltung (abhängig vom erwarteten Datenvolumen).
- Docker und Docker Compose müssen auf dem Zielsystem installiert sein, um die Datenbank im Container auszuführen.

5.5 Voraussetzungen für Installation und Betrieb

Vor dem Download und der Installation der Lösung sollten folgende Voraussetzungen erfüllt sein:

- Docker (empfohlen Version 24.0 oder höher) und Docker Compose für die Containerisierung und Verwaltung der PostgreSQL-Datenbank.
- Git (optional, zum Klonen des Repositories).
- Java Development Kit (JDK), idealerweise OpenJDK 17, für den Betrieb des Spring Boot-Backends. Stellen Sie sicher, dass java in der Systemumgebung verfügbar ist.
- Maven (Version 3.6 oder höher), um das Backend zu bauen und auszuführen.
- Node.js (Version 16 oder höher) und npm für die Installation und den Betrieb des React-Frontends.
- Netzwerkzugang, damit alle benötigten Abhängigkeiten und Containerimages heruntergeladen werden können.
- Freie Ports 3000 (Frontend) und 8080 (Backend) auf dem Hostsystem.

5.6 Installation und Betrieb

Installationsschritte:

1. Repository klonen und Setup vorbereiten:
git clone <https://github.com/mide553/IDERHA.git>
cd IDERHA
git lfs pull
2. .env Datei erstellen
(siehe .env.example)
3. application.properties erstellen
(siehe backend/src/main/resources/application.properties.example)
4. App starten

Windows:

```
./start-app.ps1
```

Linux/Mac:

```
bash start-app.sh
```

Die Lösung ist anschließend über <http://localhost:3000> (Frontend) erreichbar. Das Backend läuft auf Port 8080.

6. Unser Projekt-Tagebuch

14. Oktober 2025: Projekt-Kick-off & Sprint

- Was ist passiert: Zum Semesterstart trafen wir uns mit unserem Supervisor Lukas Rohatsch zur Auftragsklärung. Wir analysierten die Ergebnisse aus dem Vorsemester (SS25) und legten die Erweiterungen für eHealth Insights fest.
- Diskussion: Fokus auf die API-Sicherheit. Wir entschieden uns für ein striktes JWT-Verfahren und das Hashing via BCrypt.
- Herausforderung: Klärung der Projektziele und die saubere Rollenverteilung im Team, um die kommenden 6 Sprints effizient zu nutzen.
- Highlight: Erfolgreiches Aufsetzen der REST-Endpoints und des API-Key-Managements.

4. November 2025: Sprint 2

- Aktivität: In diesem Sprint stand das Frontend im Vordergrund. Wir erstellten die ersten Mockups für das Graph-Dashboard und implementierten die Filterlogik in React.

- Herausforderung: Die Komplexität der Filterfunktionen (Alter, Diagnose, Zeiträume) war höher als gedacht, um eine flüssige User Experience zu garantieren.
- Cooler Moment: Die erste visuelle Darstellung von Testdaten in den neuen UI-Komponenten funktionierte auf Anhieb.

18. November 2025: Sprint 3

- Technischer Fokus: Integration des OMOP-Schemas und Anbindung der dezentralen Datenbanken mittels Foreign Data Wrapper (FDW).
- Herausforderung (High Complexity): Performance-Probleme bei großen Datensätzen über mehrere Container hinweg. Wir mussten die DB-Indexierung massiv optimieren.
- Highlight: Der Moment, als föderierte Queries über verschiedene Spitals-Datenbanken hinweg konsistente Ergebnisse im Dashboard lieferten.

2. Dezember 2025: Sprint 4

- Was ist passiert: Wir stabilisierten die Docker-Infrastruktur. Das gesamte System (FE, BE, DB) wurde in eine saubere Container-Pipeline überführt.
- Problem gelöst: Die Abstimmung zwischen den einzelnen Containern und der CI/CD-Pipeline erforderte mehrere Debugging-Sessions, besonders bei der Migration der SQL-Dumps via Git LFS.

16. Dezember 2025: Sprint 5

- Aktivität: Implementierung der Export-Funktionen (PDF/DOC) und Finalisierung des Hospital-Managements.
- Diskussion: Wir optimierten die Usability des Dashboards basierend auf internem Feedback, um die Bedienung für Researcher so intuitiv wie möglich zu gestalten.
- Ergebnis: Die Plattform wurde stabil in der Testumgebung bereitgestellt.

13. Jänner 2026: Sprint 6

- Was ist passiert: Der letzte Sprint stand im Zeichen der Qualitätssicherung (Unit-Tests & Security Audit) und der Vorbereitung der finalen Abgabe.
- Finaler Sprint-Abschluss: Wir trafen uns, um den finalen Upload vorzubereiten.